Survival of the Cheapest: Cost-Aware Hardware Adaptation for Adversarial Robustness

1st Charles Meyers
Responsible AI Practice
Northeastern University
Boston, USA
0000-0002-1277-9811
cmeyers@cs.umu.se

2nd Mohammad Reza Saleh Sedghpour
DoiT
Lund, Sweden
0000-0002-0751-9695

3rd Erik Elmroth
dept. Computer Science
Umeå University
Umeå, Sweden
0000-0002-2633-6798

4th Tommy Löfstedt
dept. Computer Science
Umeå University
Umeå, Sweden
0000-0001-7119-7646

***Abstract*—Deploying adversarially robust machine learning systems requires continuous trade-offs between robustness, cost, and latency. We present an autonomic decision-support framework providing a quantitative foundation for adaptive hardware selection and hyper-parameter tuning in cloud-native deep learning. The framework applies accelerated failure time (AFT) models to quantify the effect of hardware choice, batch size, epochs, and validation accuracy on model survival time. This framework can be naturally integrated into an autonomic control loop (monitor–analyse–plan–execute, MAPE-K), where system metrics such as cost, robustness, and latency are continuously evaluated and used to adapt model configurations and hardware selection. Experiments across three GPU architectures confirm the framework is both sound and cost-effective: the Nvidia L4 yields a 20% increase in adversarial survival time while costing 75% less than the V100, demonstrating that expensive hardware does not necessarily improve robustness. The analysis further reveals that model inference latency is a stronger predictor of adversarial robustness than training time or hardware configuration.**

***Index Terms*—adversarial robustness, hardware-aware adaptation, survival analysis, self-adaptive systems**

I. Introduction

Cloud-native deployments of machine learning with deep neural networks are increasingly common in safety-critical classification tasks, with applications ranging from medical imaging [24] to aviation [38] and from security [5], [40], [56] to self-driving cars [11]. Statistical learning theory [51], [57] provides no guarantees about the generalisation performance of deep neural networks due to the massive number of tunable parameters. To overcome this, neural networks need large amounts of data [8], [21] to train ever-larger models [21], which has yielded increasingly marginal gains on validation accuracy [53]. It is also clear that reaching safety-critical standards using validation accuracy would require an infeasibly large data set [41].

Modern neural networks are massive, from AlexNet's 60M parameters [3] to Mamba's 8 billion [58], and now rank among the largest consumers of data-centre power [47]. At these scales, meeting even the weakest safety-critical threshold ($[10^{-12}, 10^{-15}]$ per second failure rate [1], [7], [16], [39]) would require billions of validation samples per model change — computationally infeasible [41].

In this work, we propose an autonomic decision-support framework based on accelerated failure-time (AFT) methods to predict model performance across hardware configurations and estimate deployment cost.

A. Classifiers

We consider ML classifiers, $K(x;\theta)$, with parameters θ, for a mini-batch, x, of size N with corresponding true labels y, and predictions $\hat{y} = K(x;\theta)$. A loss function, $L(y,\hat{y})$, quantifies the prediction error.

Due to model complexity, large-scale ML training typically uses mini-batch stochastic gradient descent (SGD), which updates parameters using mini-batch gradients over multiple epochs,

$$\theta^{(i+1)} = \theta^{(i)} - \eta^{(i)} \nabla_{\theta^{(i)}} L\big(y, K(x, \theta^{(i)})\big), \tag{1}$$

where $\eta^{(i)}$ is the learning rate and the gradient is computed over the mini-batch.

Hardware differences (*e.g.,* VRAM) influence optimal configurations, especially when considering performance and cost. Training and inference may also run on different hardware to optimise efficiency, as some devices are specialised for training (*e.g.,* V100, P100) and others for inference (*e.g.,* L4). An effective learning rate (η) balances fast convergence and accuracy, but in cloud environments, hardware variability requires empirical tuning. Since compute is billed per hour, optimising training time is also cost-critical.

B. Adversarial Attacks

In this work, we focus on *evasion attacks*, where the attacker perturbs the input at inference time to induce a misclassification. Formally, *adversarial success* or *failure* is one in which

$$K(x;\theta) = \hat{y} \neq \hat{y}_a = K(x+\varepsilon;\theta), \tag{2}$$

where ε is added adversarial noise, bounded as $0 < \|\varepsilon_i\| \leq \varepsilon^*$. Additionally, one can measure the accuracy of the model when tasked with these adversarial samples, $x+\varepsilon$, giving a metric called *adversarial accuracy*, which is Eq. 4 calculated on the perturbed samples, *i.e.*, on $x+\varepsilon$. These perturbations may represent malicious evasion attacks or naturally occurring

input deviations, but the objective is not to model a specific attacker. Instead, these attacks provide an efficient mechanism for measuring how quickly a model fails as input conditions deviate from the training distribution.

One of many possible attacks is called the *fast gradient method* [25]. It works by applying noise to a set of samples, x, to generate adversarial examples, x_a, such that,

$$x_a = x + \eta \cdot \operatorname{sign}\Big(\nabla_x L\big(y, K(x,\theta)\big)\Big). \tag{3}$$

Although presented here as a representative example, this attack is used throughout this work. The methods and analysis, however, generalize to any adversarial attack that generates perturbed inputs and can induce a failure.

C. Related Work

Selecting hyper-parameters for adversarially robust models is considerably more challenging than for conventional training because benign accuracy and adversarial robustness are often competing objectives [14], [22], [41]. Duesterwald et al. [23] demonstrated that adversarial training is highly sensitive to hyper-parameter selection and that no single configuration performs well across datasets or robustness objectives. In particular, robustness verification shown to be NP-hard [14].

Our work differs from previous robustness-oriented HPO studies in two important respects. First, rather than optimising solely for benign and adversarial accuracy, we additionally optimise for training efficiency. This is particularly important in practical deployments where the optimal batch size is constrained by the available GPU memory, numerical precision, and model size. Second, we focus on identifying hardware-aware training configurations that minimise wall-clock training time while maintaining competitive benign and adversarial performance.

This paper is complementary to the TRASHFIRE framework [42], which proposes survival models (detailed in the next section) for evaluating the cost-efficacy of different models and attacks.

Whereas TRASHFIRE justifies and defines training-time heuristics that generalise across experiments, this work formulates hyper-parameter selection as a multi-objective Bayesian optimisation problem across hardware configurations while measuring real-world cost in terms of time, power, and money. Specifically, we employ the Tree-structured Parzen Estimator because it has been shown to reach competitive Pareto-optimal solutions in hundreds rather than thousands of trials compared with evolutionary multi-objective optimisers such as CMA-ES and NSGA-II [2], [46], [59].

D. Contributions

To tackle the problems of scaling validation to safety critical levels, we present a methodology (Section III) that:

- Provides a quantitative decision-support component for self-adaptive systems, showing how survival models allow model builders to derive cost and robustness estimates across the feasible hyper-parameter space.
- Demonstrates a scalable and effective method to train a model while simultaneously estimating the effect of various hyper-parameters.
- Measures the power and monetary cost of deploying a model across different hardware architectures to model the trade-offs between deployment hardware and robustness.
- Provide a fast rejection method for eliminating bad candidate models (Equation 8).
- Demonstrates that "inference-only" GPUs are both cheap and effective for large numbers of datasets and many classes of models.

This paper focuses on the Monitor, Analyse, and Plan phases of MAPE-K; live Execute-phase integration with a running autonomic system will be addressed in future work.

II. Survival Analysis for Cost-Aware Robustness Estimation

AFT models are statistical models used to analyse multivariate effects on the observed failure rate to predict the time-to-failure across a wide variety of circumstances [12], [32], mapping the relationship between model tuning parameters and performance to provide an analytical foundation for runtime hardware selection in self-adaptive systems. The section below first discusses the drawbacks of using accuracy as a measure of failure probability, then motivates a move towards time-domain estimates of failure rate, and then outlines methods for comparing these time-domain models to find one that fits best.

A. Accuracy

Accuracy measures the frequentist probability of a failure. Throughout, we use the terminology *benign* accuracy to refer to the performance on a data set using unperturbed data and *adversarial* to refer to the performance in the presence of additive adversarial noise that is intended to confuse the model. The accuracy, λ, is defined as

$$\lambda := \text{Accuracy} := 1 - \frac{\text{False Classifications}}{\text{Total Classifications}}, \tag{4}$$

which is generally assumed to indicate the rate of successes in real-world data sampled from the same distribution as the training data [55]. However, it ignores run-time cost [8], [21] and adversarial noise [17]. Furthermore, adversarial counter-examples consistently expose the benign accuracy as optimistic [10], [13], [14], [33], [41] in the presence of small amounts of noise [33].

B. Failure Rate

The failure rate refers to the percentage or proportion of examples that cause the targeted ML model to misclassify or produce incorrect outputs [41]. To encompass the cost of a particular model or attack, the proposed methodology considers failures to be a function over some time interval (*e.g.,* inference time, attack generation time, training time, with parameters, θ), so that the failure rate is the average time until a failure in a time interval around time, t, such that

$$h_\theta(t) := \lim_{\Delta t \to 0} \frac{p\big(\text{False Classification in } (t, t+\Delta t] \mid \theta\big)}{\Delta t},$$

where $p\big(\text{False Classification in } (t, t+\Delta t] \mid \theta\big)$ is the probability of a false classification within a small (half-open) interval around time t, given a particular set of model parameters, θ, the Δt is a time interval length, and t is a point in time.

C. AFT Models

AFT models are widely used in industrial, medical, and risk-mitigation contexts [12], [32] to model covariate effects on expected time-to-failure; here we apply them to ML by inducing adversarial failures to measure generalisation performance under varying hardware configurations and model/attack hyper-parameters.

We model the *survival time*, $S_\theta(t)$, as a function of time, t, and some set of model parameters, θ, such that,

$$S_\theta(t) = p(T > t \mid \theta) = \exp\left(-\int_0^t h_\theta(u)\, du\right)$$

where $p(T > t \mid \theta)$ is the probability that a model has not failed by time t ("survives" beyond time t). The expected survival time is

$$\mathbb{E}_{S_\theta}[T] = \int_0^{t^*} S_\theta(u)\, du, \tag{5}$$

where t^* is the latest time observed in the survival data. However, modelling $S_\theta(t)$ requires a choice of modelling function for S_θ, a function of u. The Log-Logistic, Log-Normal, and Weibull functions are widely used alternative modelling functions [32], [42]. For each trial, one can measure the attack generation time to define the time interval and the adversarial accuracy to estimate the number of failures and successes in that time interval.

1) Survival Time: By using adversarial samples (see Equation 2), The likelihood of those failures depends on the amount of adversarial noise, ε, which is included in the resultant AFT model as a parameter. In the language of accelerated failure time models, this can be expressed in terms of the accelerated failure time assumption [32]

$$S_\theta(t) = S_0\left(\frac{t}{\phi_\theta(x)}\right), \tag{6}$$

where ϕ_θ is the acceleration factor, described by the joint effect of the covariates, such that

$$\phi_\theta(x) = \exp\left(\theta_0 x_0 + \theta_1 x_1 + \cdots + \theta_n x_n\right),$$

where $x = (x_0, \ldots, x_n)$ is a vector of covariates, $\theta = (\theta_0, \ldots, \theta_n)$ describe the fitted parameters. That is, as attack strength increases the time-to-failure decreases.

Any attack that produces a measurable degradation in model performance over time can be modelled using an AFT formulation, assuming there is a multiplicative or additive relationship between attack strength and time-to-failure [42].

D. Choosing the best AFT Model

To choose a best-fit from a number of possible AFT functions, one should prepare the collected metrics and scores and then compare them using, *e.g.,* Akaike Information Criterion (AIC), Bayesian Information Criterion (BIC), or Concordance, as per the best practices for this methodology [12], [32]. For AIC and BIC, that means choosing the model giving the smallest value. Concordance, however, is a number between 0 and 1 that quantifies the degree to which the survival time is explained by the model, where a 1 reflects a perfect explanation [32], 0.5 reflects random chance, and 0 reflects a model that is perfectly wrong (a sign error). By evaluating $\mathbb{E}_{S_\theta}[T]$ under extreme perturbations, one can test the model and minimise the number of evaluated samples [12], [32] rather than relying on $> 10^{12}$ validation samples, as required by IEC61508 [16]. The integrated calibration index (ICI) as well as the error at the 50th percentile E50 [6] are then calculated. These are the mean absolute difference between observed and predicted probabilities and the median absolute difference between observed and predicted probabilities, respectively [6]. Additionally, the data were split into a training set that was used to fit the model (80%) and an unseen validation set (20%), and the concordance, ICI, and E50 were measured for both.

III. Cost and Robustness Metrics for Hardware Adaptation

Building on the survival analysis in Section II-C, this section defines the cost and robustness metrics that inform the model hyper-parameter selection process. Cost is considered across three dimensions: computational efficiency (TRASH score [42]), monetary deployment cost, and energy consumption. Together these metrics allow us to quantify the cost of training, deployment, and attacking a given model [16] on a given hardware configuration: the benign and adversarial accuracies (see Equations 2 and 4), the model training time, t_t, the model inference time or latency, t_i, the attack generation time, t_a, the cost per hour for a particular hardware, C, as well as the power consumption, P, of each tested model and attack.

A. Accuracy

Under the TRASHFIRE framework [42], adversarial accuracy serves as a measure of survival time across a specified time period, as outlined in Section II-C.

B. Training Time

The training time, T_t, is the time it takes to evaluate n samples, where t_t is the training time per sample. It is defined as

$$T_t := t_t \cdot n \cdot m,$$

where m is the number of epochs.

C. Latency

Latency is the time it takes to respond to a query. We assume that latency per sample is

$$T_i := t_i \cdot n,$$

which will be driven by the memory bandwidth (measured in bits/second) of a given CPU or GPU and the size [52] and complexity [28] of a given neural network architecture.

D. Attack Generation Time

We distinguish between (Equation 7) the computational cost of generating (both failed and successful) attacks and (Equation 8) the stochastic time until a model failure occurs under attack.

Assuming a constant attack generation time per sample, the empirical average attack time over n samples is

$$t_a := \frac{T_a}{n}, \tag{7}$$

where T_a denotes the total time required to generate n adversarial samples.

In contrast, the time until a successful attack induces model failure is a random variable. Prior work [42] models this quantity using survival analysis. Treating model and attack parameters as covariates θ, the expected time-to-failure can be expressed using an accelerated failure time (AFT) model such that

$$t'_a \approx \mathbb{E}_{S_\theta}[T] = \int_0^{T_a} S_\theta(t)\, dt. \tag{8}$$

We denote this expected survival time by t'_a, emphasizing that it captures the expected time until adversarial failure. Accordingly, t_a and t'_a represent fundamentally different quantities and are not directly comparable without additional assumptions linking attack generation and success processes.

E. TRASH Score

With an estimate of the expected survival time in hand, we quantify the cost-normalised failure rate, or the ratio of training time to attack time. Assuming that the cost scales linearly, as discussed above, the model builder's cost is proportional to the training time ($C_t \propto t_t$), and the attacker's cost is proportional to the attack time, which is approximately the expected survival time ($C_a \propto t'_a \approx \mathbb{E}_{S_\theta}[T]$). Using the definition of ε from Equation 2 and definition of t'_a from Equation 8, we can express the cost of failure in adversarial terms as a function of per-sample inference time (t_t) and per-sample attack time (t_a) [42],

$$\text{TRASH} \approx \frac{t_t}{\mathbb{E}_{S_\theta}[T]} = \frac{t_t}{t'_a}, \tag{9}$$

where a value larger than one indicates a model that is cheaper to break than it is to train.

F. Monetary Deployment Cost

Furthermore, we approach the cost of deployment at two scales. Firstly, we consider the cloud-rental scale, where a small business might test and deploy a model using, *e.g.,* the Google Cloud Platform (GCP) compute costs as a measure of total cost. However, at a certain scale or with certain applications, it is more appropriate to talk about cost in terms of power (*e.g.,* to deploy a self-driving car with a useful operating range).

Finally, we define metrics that provide an efficient way to minimise the latency and cost of deployment, and to maximise the generalised performance of a model. We define the training cost as

$$C_t = C_h \cdot T_t,$$

where C_h is the cost per unit time for the hardware, T_t is the training time. Inference and attack costs follow analogously, denoted with i and a subscripts respectively.

G. Power

The power consumption for a particular piece of hardware, P_h, measured in Watts (Joules per second), can be thought of similarly such that the total power consumption of model training, inference, and attack is denoted with t, i, and a subscripts respectively. In this work, power was monitored in real time using KEPLER [4], providing the energy cost metrics that inform the MAPE-K loop.

IV. Experiments

This section details the implementation of the methodology in Sections II–III to estimate the cost, power consumption, and adversarial robustness of ML models across the hyper-parameter and hardware search space, enabling rapid rejection of poor configurations and informed planning for self-adaptive cloud-native systems.

A. Cloud Platform and Hardware

To conduct the experiments and have access to different types of hardware, we utilised GCP. Six virtual machines running Container-Optimised OS provided by GCP constituted the test-bed. Using Google Kubernetes Engine 1.27.3 and Containerd 1.7.0, a cluster consisting of six worker nodes was created. Three worker nodes were responsible for running the monitoring platforms — Prometheus 2.47.2 and Grafana 10.2.0. These nodes were of the "e2-medium" instance type provided by GCP. In total, three GPU architectures were used — the Nvidia P100, V100, and L4. For P100 and V100 GPUs, the "n1-standard-2" type was used for the nodes and for L4 GPUs the "g2-standard-4" was used.

To assess the energy consumption of the experiments, KEPLER was deployed on each node to measure the power consumption of each experiment [4]. KEPLER collects per-pod energy metrics within the Kubernetes cluster and exports them as Prometheus metrics [50]. Finally, all of the experiments were restricted to a $1,000 budget (a research grant from Google). Approximately 10% was used for development and 90% for the evaluations. Over the course of the evaluations, 6% of the budget went to storage, 6% to monitoring, and the remaining 88% went to the GPUs.

B. AFT Models

For each hardware configuration and dataset (see Section IV-C), the Tree-Parzen Estimator (TPE) algorithm was used to select the parameters [46], [62], and it was iterated for 1,000 trials. We used three optimisation criteria: benign accuracy, training time, and adversarial accuracy, seeking to

maximise both benign and adversarial accuracy while minimising training time (and therefore deployment cost). We selected a set of parameters as per the TPE algorithm and trained on 80% of the samples for each of the MNIST, CIFAR10, and CIFAR100 datasets. Of the remaining samples, 100 were withheld to be attacked and used to evaluate the adversarial accuracy. For each dataset, we tested this on 10 random splits of the data to create 10 unique train/validation pairs. For each trial, we recorded attack generation time, model training time, model inference time, benign accuracy and adversarial accuracy, and the size of the training set, validation set, and attack set. Using these values, we fit an AFT model to the number of failures (indicated by accuracy and sample size) and the attack generation time after adding categorical variables for the dataset and hardware device.

C. Datasets

The AFT models were evaluated using the MNIST [20], CIFAR10 [34], and CIFAR100 [34] datasets, chosen primarily for their standardised use in adversarial analysis [14], [17], [37], [43] and decades of experimental results. Before training, we centred and scaled the data so that the attack distance would be comparable for all tested datasets. Furthermore, to reduce the complexities of system overhead, distributed or federated training, and the effect of shared cloud environments, we restricted ourselves to datasets that were small enough to reside entirely within GPU memory with the model— testing these more complicated configurations is left to future work though the general methodology would remain unchanged.

D. Models

The evaluations were restricted to a single model. Primarily, this was done to meet the budgetary constraints since evaluating more models would mean evaluating fewer pieces of hardware. As exact verification of adversarial robustness is computationally intractable—being NP-hard even for common classes of neural networks [14], [48]— therefore, we tested each model and attack over a large hyper-parameter space. So, we sampled learning rates in $[10^{-6}, 1]$, batch sizes in $[1, 10^5]$, and epochs in $[1, 50]$ for MNIST and CIFAR10 on the P100 and V100. For CIFAR100, the range of the tested epochs was increased to be in $[1, 100]$. The Feature Squeezing defence [61] was used to evaluate the efficacy of different bit-depths on the L4 hardware, as provided by IBM's adversarial robustness toolbox [44] with bit depths in $[4, 8, 16, 32, 64]$ which casts the inputs into `pytorch`-compatible arrays. Feature Squeezing was applied only to the L4 because that GPU's tensor cores natively execute 8-bit operations; applying the same bit-depth reduction to the P100 or V100 would instead emulate 8-bit arithmetic in software on 32-bit silicon, which does not reflect those GPUs' actual operating mode. Model parameters were chosen using the `optuna` optimisation framework, the configuration was handled by `hydra`, and `dvc` was used to ensure reproducibility and aid in collaborative development.

E. Attacks

To examine the effect of model parameters at run-time, evasion attacks, which attack the model at the prediction stage, were examined.

The goal of using these attacks is to model the effect of noise on model performance rather than imagine a hypothetical Prior research [41], [42] has shown that the Fast Gradient Method (see Eq. 3) is consistently the most effective at inducing a large number of failures in a small amount of time. To evaluate the effect of adversarial noise (or lack thereof) on the samples, the noise levels were varied $0 < \|\varepsilon_i\| \leq 1$, sampled randomly from a uniform distribution. This was done using the `adversarial-robustness-toolbox` package maintained by a team at IBM [44].

F. GPU Configurations

Several hardware configurations were tested that had various hourly costs, peak power demands, and theoretical memory bandwidths. The V100 was chosen as a baseline, since it is routinely used in the literature [54], [60]. The P100 architecture comes from the same line of server-grade GPUs, but from an older generation. The L4, however, is advertised as a machine built for inference, not training, relying on a smaller number of bits per tensor core. Consequently, the number of operations per second depends on the bit depth of the data and model weights, with peak numbers outlined in Table I for 8-bit inputs. The rental cost of the hardware, measured in United States Dollars per hour, indicates the operating cost of a given model. To calculate this, the price per hour from each cloud service pricing page was used [26], [27], and the cost of training (C_t) and the cost of inference (C_i) were calculated from the cost of hardware (C_h), the training time (T_t), and the inference time (T_i).

G. Survival Analysis

In addition to the optimisation criteria of benign/adversarial accuracy and training time, prediction times, attack times, power consumption, batch size, attack noise, and number of epochs were also collected to be used as covariates in the AFT model, $S_\theta(t)$. To fit the AFT model and to plot the effect of the covariates, the `lifelines` Python package was used [18]. The Weibull, Log Logistic, and Log Normal AFT models (see Section II) were compared as per Section II-D.

V. Results and Discussion

The experiments demonstrate that the proposed methodology provides reliable and cost-effective estimates of adversarial robustness across diverse hardware configurations.

A. Accuracy

Figure 1 shows the benign (left) and adversarial (right) accuracies for all datasets and hardware. It demonstrates little to no change in accuracy or adversarial accuracy, regardless of hardware. The benign accuracy decreases with difficulty (CIFAR10 *vs.* MNIST) or with the number of classes (CIFAR10 *vs.* CIFAR100). For all three datasets, the adversarial accuracy

	V100	P100	L4
Cost (USD/hour)	2.55	1.60	0.81
Power (Watts)	250	250	72
Memory Bandwidth (GB/s)	900	732	300

TABLE I

HARDWARE SPECIFICATIONS FOR THE TESTED GPUS. THE SPECIFICATIONS WERE RETRIEVED FROM NVIDIA'S WEBSITE AT THE FOLLOWING LINKS: V100 DATASHEET, P100 DATASHEET, AND L4 DATASHEET. PRICES WERE RETRIEVED FROM GOOGLE CLOUD PLATFORM FOR THE `EUROPE-WEST4` REGION ON 3 DECEMBER 2023.

becomes the reciprocal of the number of classes (*i.e.*, the accuracy we would expect with random data), demonstrating the efficacy of the attack outlined in Section I-B.

B. Time, Power, and Monetary Cost

The power consumption during training, inference, and attack generation for all hardware and datasets is illustrated in Figure 2. As expected, it tracks monetary cost (Figure 3) closely — since power is the predominant operating cost for data centres [19], cloud billing and power draw are structurally coupled by design.

The monetary cost for each dataset and each piece of hardware is shown in Figure 3. For all three datasets across all three pieces of hardware, the cost of training on a single sample often exceeds the cost of attacking a single sample. In the best-case scenario, they are comparable, but attacks consistently succeed with only 100 samples (Figure 1) while model training requires orders of magnitude more. Together, the power and cost measurements in Figures 2–3 constitute Monitor-phase observables that a MAPE-K Analyse phase can use to rank hardware configurations by cost-effectiveness.

C. AFT Models

Table II shows the performance metrics for all three AFT models, as outlined in Section II-C. AIC and BIC are measures of goodness-of-fit, with a smaller value being preferred. Concordance (Conc) is a value between 0 and 1 that reflects what proportion of events (failures) can be explained by the model. ICI measures the average error between a cubic-spline and the model and E50 measures the median error between the spline and the model. These are measured on both the train and validation sets of the measured data, with the latter being denoted "Val". The concordance is both strong (> 0.5), similar for all three AFT models, as well as consistent across both the train and validation sets. We observed no more than 1% mean absolute error in the probabilities ICI and no error in the median probability E50 across all three AFT models. Figure 4 shows the log-scale coefficients for the AFT model. It shows that epochs, batch size, and training time have no effect on the survival time. However, we can clearly see that inference time is as strong an indicator as the attack noise distance, revealing that model speed is nearly as important as the magnitude of the attack (ε). Furthermore, we see that benign accuracy is negatively correlated with the survival time, confirming previous assertions that robustness ($S_\theta(t)$) is inversely related to benign accuracy (which is the standard indicator of generalisation performance) [14].

Figure 4 further shows that hardware choice has a relatively small effect on robustness — the L4 increases adversarial survival time by approximately 20% and the P100 decreases it, both relative to the V100 — despite the much larger disparity in cost outlined in Table I.

D. Why Cost Matters

Security analyses routinely assume optimistic scenarios for both attackers and defenders. In cryptography, these idealised attack and defence models are used to determine whether a scheme is computationally feasible to break [30], [36]. Likewise, if the cost of training a model greatly exceeds the cost of attacking it, then the model is effectively "broken" and should be discarded [42]. Figure 5 shows the TRASH score (Eq. 9), which measures the per-sample ratio of training to attack time.

Despite its lower memory bandwidth (Table I), the L4 achieves superior cost (Table I) and robustness (Figure 4) because its tensor cores natively execute 8-bit operations, matching the 8-bit MNIST, CIFAR10, and CIFAR100 datasets. The additional precision of the P100 and V100 provides no useful information for these workloads. This result directly supports the Analyze phase of a MAPE-K loop by enabling self-adaptive systems to select the least expensive hardware configuration without sacrificing robustness and to discard models that are cheaper to attack than to train.

E. Advantages of the Proposed Methodology

Unlike conventional train/validation evaluation, whose precision is limited by the number of samples, the precision of survival-time estimates is determined by hardware clock resolution. Demonstrating the failure rates required by safety standards such as IEC 61508 (*e.g.,* one failure in a million) would require millions of validation samples and repeated data collection after each software change [16]. In contrast, the proposed AFT model achieves an average error of approximately 1% using only 10 sets of 100 samples (Table II).

Because AFT models require relatively few samples, they can act as lightweight unit tests for ML components, estimating marginal risk per IEC 61508 without full-system integration or deployment [16], [35], [49]. Their strong predictive performance on unseen hyper-parameter configurations can also reduce the search space by eliminating candidates unlikely to improve survival time. Finally, the resulting cost and robustness estimates naturally support the planning stage of a MAPE-K control loop [31], making the methodology suitable as a quantitative decision-support component for self-adaptive cloud-native ML systems.

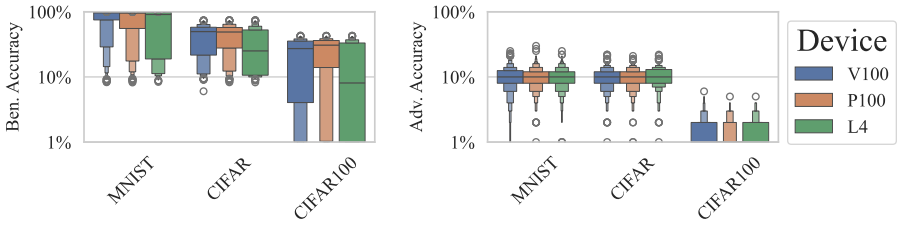

Fig. 1. Benign and adversarial accuracy across all hardware and datasets for all 1,000 trials using plots that depict the distribution of the first axis values using the width of the plot. Each colour is a different device and the datasets are displayed along the first axis. Outliers are denoted with a white dot.

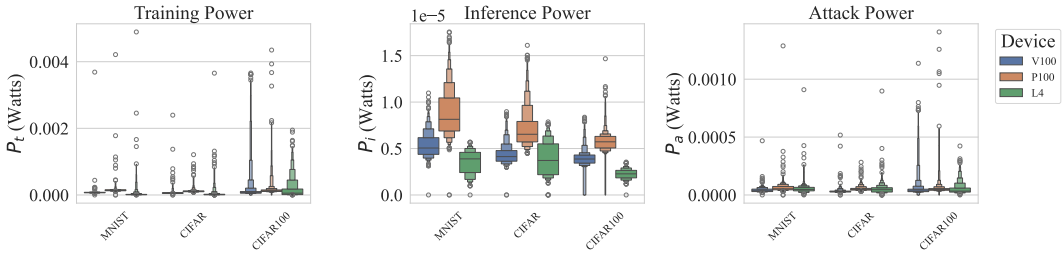

Fig. 2. Here we depict the power consumption during training, inference, and attack generation for all hardware and datasets for all 1,000 trials using plots that depict the distribution of the second axis values using the width of the plot. The power per sample was assumed to be uniform across the batch of samples for each training, inference, or attack measurement. Each colour is a different device and the datasets are displayed along the first axis. Outliers are denoted with a white dot. For these plots, the second axes have been scaled by the number of samples for the sake of comparison.

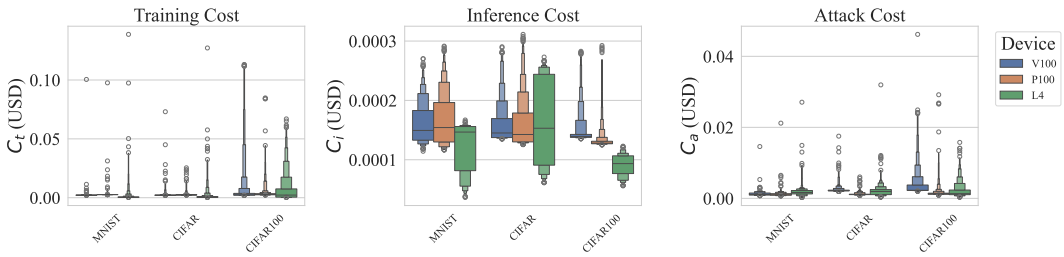

Fig. 3. This depicts the monetary cost during training, inference, and attack generation for all hardware and datasets for all 1,000 trials using plots that depict the distribution of the second axis values using the width of the plot. The cost per sample was assumed to be uniform across the batch of samples for each training, inference, or attack measurement. Each colour is a different device and the datasets are displayed along the first axis. Outliers are denoted with a white dot. For these plots, the second axes have been scaled by the number of samples for the sake of comparison.

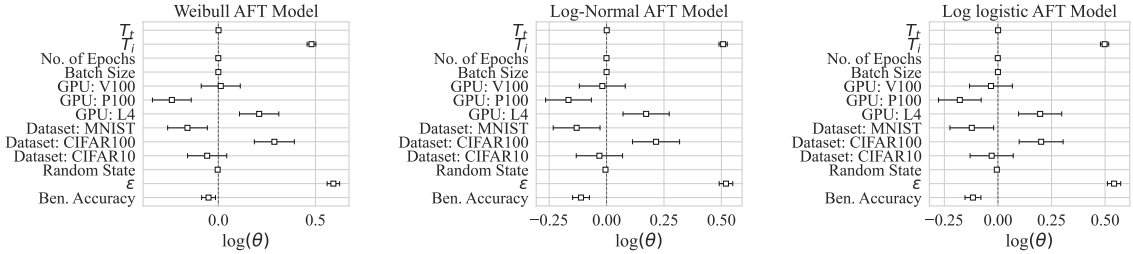

Fig. 4. Coefficients of the Covariates for the Weibull, Log-Normal, and Log-Logistic AFT models. Here, "Random State" is used as a control variable that should be (and is) close to 0. A positive value indicates that a covariate increases the survival time and a negative value indicates that a covariate decreases the survival time. The symbols T_i and T_t refer to training and inference time for all samples, while *Ben. Accuracy* refers to the benign accuracy (accuracy on the un-altered samples), and ε is the noise distance.

TABLE II
COMPARISON OF AFT MODELS ACROSS ALL HARDWARE AND DATASETS.

	AIC	BIC	Conc	Val Conc	ICI	Val ICI	E50	Val E50
Weibull	$-2.11 \cdot 10^4$	$-2.11 \cdot 10^4$	0.84	0.83	0.00	0.01	0.00	0.00
Log Logistic	$-2.18 \cdot 10^4$	$-2.18 \cdot 10^4$	0.84	0.83	0.01	0.01	0.00	0.00
Log Normal	$-2.25 \cdot 10^4$	$-2.25 \cdot 10^4$	0.84	0.83	0.01	0.00	0.00	0.00

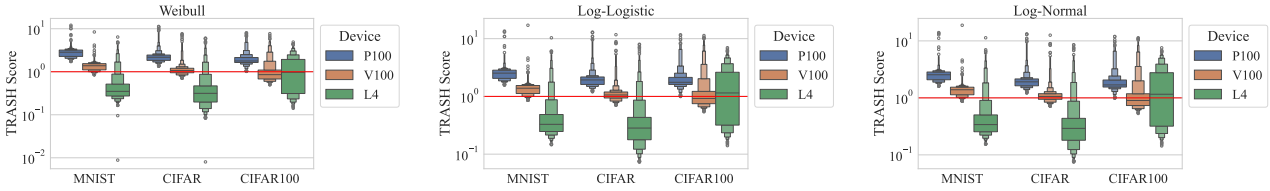

Fig. 5. The *training rate and survival heuristic* score (TRASH score) that depicts the per-sample ratio of training time to attack time for each of the tested AFT models. If this value is greater than one (the red line), then the model can be discarded as ineffective. The first axis shows each dataset, the second axis shows the TRASH score and each GPU model is given its own colour. Outlier scores are depicted with a white dot.

VI. SCOPE AND LIMITATIONS

Although this work focuses on adversarial robustness, the proposed cost and survival analysis framework is general and can be applied whenever there is a multiplicative relationship between attack strength and model failure.

Timing measurements were designed to minimise jitter and account for GPU parallelism by assuming a uniform time-to-failure across samples within each benign or adversarial evaluation. While some samples or classes are likely easier to attack than others, we assume these effects average over the 100 attack samples. Although modelling class- or sample-specific failure rates is outside the scope of this work, it could reduce some of the remaining unexplained variance. Nevertheless, the strong agreement across AFT models (Table II) suggests that the uniform-time assumption has little practical impact.

To minimise confounding factors, we restricted our experiments to models and datasets that fit entirely within GPU memory, isolating the effects of model configuration and hardware from secondary bottlenecks such as storage I/O, PCIe transfers, and distributed communication. While these factors can significantly affect latency and cost in production cloud deployments, they can be incorporated as additional covariates in the proposed survival analysis framework. The 24 GB VRAM of the L4 is sufficient for standard 8-bit vision benchmarks (*e.g.,* MNIST, CIFAR10, and CIFAR100); indeed, at the time of publication, 98.9% of publicly available Kaggle datasets are smaller than 24 GB [29]. Extending the methodology to distributed and federated settings remains future work.

The attack and training batch sizes were matched in every trial so that attack timing reflected typical deployment conditions. Since attacks operate on fewer samples than training, the additional parallelisation overhead likely causes attack efficacy to be slightly underestimated relative to benign measurements.

Finally, hyper-parameter optimisation was restricted to a single evasion attack (Eq. 3) due to computational constraints. However, the proposed AFT-based methodology naturally extends to other attack classes, including evasion, extraction, inversion, and poisoning attacks [9], [10], [15], [45].

VII. CONCLUSIONS

We presented an autonomic decision-support framework that applies survival analysis to cloud-native ML, enabling rapid, cost-efficient evaluation of hyper-parameter choices under adversarial noise. The results show that adversarial robustness is influenced far more by attack parameters than by reduced training times on newer or larger hardware.

The experiments demonstrate that the methodology is both sound and economical. Although GPU rentals accounted for 88% of the cloud budget, replacing 32-bit data-centre GPUs (V100 and P100) with the 8-bit-optimised L4 reduced GPU costs by 75% while improving both robustness and cost-effectiveness for standard image classification tasks. KEPLER proved similarly efficient, accounting for only 6% of the total budget. AFT models achieved concordance scores above 0.83 on held-out data, confirming survival analysis as a reliable estimator of adversarial robustness. Attack cost consistently remained below training cost across all datasets and hardware, while the fitted coefficients identified inference latency (T_i)—rather than training time, model complexity, or epoch count—as the dominant predictor of robustness.

The coefficients also reinforce the well-known inverse relationship between benign accuracy and adversarial robustness. Even after accounting for GPU bandwidth (Table I), the L4 remained the most robust and cost-effective platform (Figure 5), despite being marketed primarily for inference.

Future work should extend the framework to black-box, poisoning, and membership inference attacks, validate its integration within live MAPE-K control loops, and evaluate distributed and federated deployments representative of modern large-scale ML systems.

REFERENCES

[1] PV Srinivas Acharyulu and P Seetharamaiah. A framework for safety automation of safety-critical systems operations. *Safety Science*, 77:133–142, 2015.

[2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2623–2631, 2019.

[3] Md Zahangir Alom, Tarek M Taha, Christopher Yakopcic, Stefan Westberg, Paheding Sidike, Mst Shamima Nasrin, Brian C Van Esesn, Abdul A S Awwal, and Vijayan K Asari. The history began from alexnet: A comprehensive survey on deep learning approaches. *arXiv preprint arXiv:1803.01164*, 2018.

[4] Marcelo Amaral, Huamin Chen, Tatsuhiro Chiba, Rina Nakazawa, Sunyanan Choochotkaew, Eun Kyung Lee, and Tamar Eilam. Kepler: A framework to calculate the energy consumption of containerized applications. In *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*, pages 69–71, 2023.

[5] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don'ts of machine learning in computer security. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3971–3988, 2022.

[6] Peter C Austin, Frank E Harrell Jr, and David van Klaveren. Graphical calibration curves and the integrated calibration index (ICI) for survival models. *Statistics in Medicine*, 39(21):2714–2742, 2020.

[7] C Warren Axelrod. Applying lessons from safety-critical systems to security-critical software. In *2011 IEEE Long Island Systems, Applications and Technology Conference*, pages 1–6. IEEE, 2011.

[8] Alexandre Bailly, Corentin Blanc, Élie Francis, Thierry Guillotin, Fadi Jamal, Béchara Wakim, and Pascal Roy. Effects of dataset size and interactions on the prediction performance of logistic regression and deep learning models. *Computer Methods and Programs in Biomedicine*, 213:106504, 2022.

[9] Battista Biggio, Igino Corona, Davide Maiorca, Blaine Nelson, Nedim Šrndić, Pavel Laskov, Giorgio Giacinto, and Fabio Roli. Evasion attacks against machine learning at test time. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2013, Prague, Czech Republic, September 23-27, 2013, Proceedings, Part III 13*, pages 387–402. Springer, 2013.

[10] Battista Biggio, Blaine Nelson, and Pavel Laskov. Poisoning Attacks against Support Vector Machines. *arXiv:1206.6389 [cs, stat]*, March 2013.

[11] Mariusz Bojarski, Davide Del Testa, Daniel Dworakowski, Bernhard Firner, Beat Flepp, Prasoon Goyal, Lawrence D Jackel, Mathew Monfort, Urs Muller, Jiakai Zhang, et al. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.

[12] Mike J Bradburn, Taane G Clark, Sharon B Love, and Douglas Graham Altman. Survival analysis part II: multivariate data analysis–an introduction to concepts and methods. *British journal of cancer*, 89(3):431–436, 2003.

[13] Tom B Brown, Dandelion Mané, Aurko Roy, Martín Abadi, and Justin Gilmer. Adversarial patch. *arXiv:1712.09665*, 2017.

[14] N. Carlini and D. Wagner. Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 39–57, 2017.

[15] Christopher A Choquette-Choo, Florian Tramer, Nicholas Carlini, and Nicolas Papernot. Label-only membership inference attacks. In *International conference on machine learning*, pages 1964–1974. PMLR, 2021.

[16] International Electrotechnical Commission. Iec 61508 safety and functional safety, 2010.

[17] Francesco Croce and Matthias Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *International conference on machine learning*, pages 2206–2216. PMLR, 2020.

[18] Cameron Davidson-Pilon. lifelines: survival analysis in python. *Journal of Open Source Software*, 4(40):1317, 2019.

[19] Miyuru Dayarathna, Yonggang Wen, and Rui Fan. Data center energy consumption modeling: A survey. *IEEE Communications surveys & tutorials*, 18(1):732–794, 2015.

[20] Li Deng. The MNIST database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine*, 29(6):141–142, 2012.

[21] Radosvet Desislavov, Fernando Martínez-Plumed, and José Hernández-Orallo. Compute and energy consumption trends in deep learning inference. *arXiv:2109.05472*, 2021.

[22] Elvis Dohmatob. Generalized No Free Lunch Theorem for Adversarial Robustness. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *PMLR*, 2019.

[23] Evelyn Duesterwald, Anupama Murthi, Ganesh Venkataraman, Mathieu Sinn, and Deepak Vijaykeerthy. Exploring the hyperparameter landscape of adversarial robustness. *arXiv preprint arXiv:1905.03837*, 2019.

[24] Bradley J Erickson, Panagiotis Korfiatis, Zeynettin Akkus, and Timothy L Kline. Machine learning for medical imaging. *Radiographics*, 37(2):505–515, 2017.

[25] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv:1412.6572*, 2014.

[26] Google. Gpu pricing. compute engine: Virtual machines (vms). google cloud.

[27] Google. Gpu pricing. compute engine: Virtual machines (vms). google cloud.

[28] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[29] Kaggle. Kaggle datasets, 2026. Accessed: 2026-04-05.

[30] Parves Kamal. A study on the security of password hashing based on gpu based, password cracking using high-performance cloud computing. Master's thesis, St. Cloud State. St. Cloud, Minnesota, USA., 2017.

[31] Jeffrey O. Kephart and David M. Chess. The vision of autonomic computing. *Computer*, 36(1):41–50, 2003.

[32] David G Kleinbaum and Mitchel Klein. *Survival analysis a self-learning text*. Springer, New York, NY, USA, 2012.

[33] Shashank Kotyan and Danilo Vasconcellos Vargas. Adversarial robustness assessment. *PloS one*, 17(4):e0265723, 2022.

[34] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. Technical report, Toronto, ON, Canada, 2009.

[35] John M Lachin. Introduction to sample size determination and power analysis for clinical trials. *Controlled clinical trials*, 2(2):93–113, 1981.

[36] Gaëtan Leurent and Thomas Peyrin. SHA-1 is a shambles: First Chosen-Prefix collision on SHA-1 and application to the PGP web of trust. In *29th USENIX security symposium (USENIX security 20)*, pages 1839–1856, 2020.

[37] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv:1706.06083*, 2017.

[38] Apoorv Maheshwari, Navindran Davendralingam, and Daniel A DeLaurentis. A comparative study of machine learning techniques for aviation applications. In *2018 Aviation Technology, Integration, and Operations Conference*, page 3980, 2018.

[39] International Standards Organization. ISO 26262-1:2011, road vehicles — functional safety. https://www.iso.org/standard/43464.html, 2018.

[40] Domingo Mery, Erick Svec, Marco Arias, Vladimir Riffo, Jose M Saavedra, and Sandipan Banerjee. Modern computer vision techniques for x-ray testing in baggage inspection. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 47(4):682–692, 2016.

[41] Charles Meyers, Tommy Löfstedt, and Erik Elmroth. Safety-critical computer vision: An empirical survey of adversarial evasion attacks and defenses on computer vision systems. *Artificial Intelligence Review*, 2023.

[42] Charles Meyers, Mohammad Reza, Tommy Löfstedt, and Erik Elmroth. A systematic approach to robustness modelling. In *Accepted for International Conference on Machine Learning and Cybernetics*, 2023.

[43] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, and Pascal Frossard. Deepfool: a simple and accurate method to fool deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2574–2582, 2016.

[44] Maria Irina Nicolae, Mathieu Sinn, Minh Ngoc Tran, Beat Buesser, Ambrish Rawat, Martin Wistuba, Valentina Zantedeschi, Nathalie Baracaldo, Bryant Chen, Heiko Ludwig, Ian Molloy, and Ben Edwards. Adversarial robustness toolbox v1.2.0. *CoRR*, 1807.01069, 2018.

[45] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. Knockoff nets: Stealing functionality of black-box models. In *Proceedings of*

the IEEE/CVF conference on computer vision and pattern recognition, pages 4954–4963, 2019.

[46] Yoshihiko Ozaki, Yuki Tanigaki, Shuhei Watanabe, and Masaki Onishi. Multiobjective tree-structured parzen estimator for computationally expensive optimization problems. In *Proceedings of the 2020 genetic and evolutionary computation conference*, pages 533–541, 2020.

[47] Matt O'Brien, Hannah Fingerhut, and The Associated Press. A.i. tools fueled a 34% spike in microsoft's water consumption, and one city with its data centers is concerned about the effect on residential supply, Sep 2023.

[48] Wenjie Ruan, Min Wu, Youcheng Sun, Xiaowei Huang, Daniel Kroening, and Marta Kwiatkowska. Global robustness evaluation of deep neural networks with provable guarantees for the hamming distance. IJCAI, 2019.

[49] Claudia Schmoor, Willi Sauerbrei, and Martin Schumacher. Sample size considerations for the evaluation of prognostic factors in survival analysis. *Statistics in medicine*, 19(4):441–452, 2000.

[50] Mohammad Reza Saleh Sedghpour and Paul Townend. Service mesh and ebpf-powered microservices: A survey and future directions. In *2022 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, pages 176–184, 2022.

[51] Shai Shalev-Shwartz and Shai Ben-David. *Understanding machine learning: From theory to algorithms*. Cambridge university press, Cambridge, UK, 2014.

[52] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[53] Chen Sun, Abhinav Shrivastava, Saurabh Singh, and Abhinav Gupta. Revisiting unreasonable effectiveness of data in deep learning era. In *Proceedings of the IEEE international conference on computer vision*, pages 843–852, 2017.

[54] Martin Svedin, Steven WD Chien, Gibson Chikafa, Niclas Jansson, and Artur Podobas. Benchmarking the nvidia gpu lineage: From early k80 to modern a100 with asynchronous memory transfers. In *Proceedings of the 11th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies*, pages 1–6, 2021.

[55] Jimin Tan, Jianan Yang, Sai Wu, Gang Chen, and Jake Zhao. A critical look at the current train/test split in machine learning. *arXiv preprint arXiv:2106.04525*, 2021.

[56] Guido Vittorio Travaini, Federico Pacchioni, Silvia Bellumore, Marta Bosia, and Francesco De Micco. Machine learning and criminal justice: A systematic review of advanced methodology for recidivism risk prediction. *International journal of environmental research and public health*, 19(17):10594, 2022.

[57] Vladimir N Vapnik. An overview of statistical learning theory. *IEEE transactions on neural networks*, 10(5):988–999, 1999.

[58] Roger Waleffe, Wonmin Byeon, Duncan Riach, Brandon Norick, Vijay Korthikanti, Tri Dao, Albert Gu, Ali Hatamizadeh, Sudhakar Singh, Deepak Narayanan, et al. An empirical study of mamba-based language models. *arXiv preprint arXiv:2406.07887*, 2024.

[59] Shuhei Watanabe. Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance. *arXiv preprint arXiv:2304.11127*, 2023.

[60] Rengan Xu, Frank Han, and Quy Ta. Deep learning at scale on nvidia v100 accelerators. In *2018 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, pages 23–32. IEEE, 2018.

[61] Weilin Xu, David Evans, and Yanjun Qi. Feature squeezing: Detecting adversarial examples in deep neural networks. *arXiv:1704.01155*, 2017.

[62] Eckart Zitzler, Joshua Knowles, and Lothar Thiele. Quality assessment of pareto set approximations. *Multiobjective optimization: Interactive and evolutionary approaches*, pages 373–404, 2008.